

Sprint 6 Structured Notes: Local Git Workflow

1. Current Progress

You are now in **Sprint 6** of the Relational Databases learning path.

The full learning path is:

Tools → Terminal → Bash → Bash Scripting → Git → PostgreSQL → SQL → Normalization, Security, Performance

So far, you have completed these stages:

Sprint 1: Tools and VS Code

You learned how to prepare your coding environment. You learned about:

- VS Code
- Code editors and IDEs
- Workspaces
- Project folders
- `index.html`
- Live Server
- Basic VS Code shortcuts
- Beginner extensions

Simple meaning: Sprint 1 helped you prepare the place where you write and manage project files.

Sprint 2: Terminal, Shell, and Command Line Basics

You learned what happens when you type commands. You learned about:

- Terminal
- Command line
- Shell
- Bash
- Command history
- Auto-complete
- Clearing the terminal screen
- Stopping a running command

Simple meaning: Sprint 2 helped you understand the environment where commands are typed and run.

Sprint 3: Bash File and Folder Commands

You learned how to manage files and folders using Bash. You learned commands such as:

- `pwd`
- `cd`
- `ls`
- `mkdir`
- `touch`
- `echo`
- `cat`
- `less`
- `mv`
- `cp`
- `rm`
- `>`
- `>>`

Simple meaning: Sprint 3 taught you how to move around folders, create files, read files, rename files, copy files, delete files, and write text into files using commands.

Sprint 4: Bash Scripting Basics

You learned how to save Bash commands inside a file and run them together. You learned about:

- Bash scripts
- `.sh` files
- Shebang
- Interpreter paths
- Variables
- Variable interpolation
- Arrays
- `for` loops
- `do` and `done`
- `echo` logging
- `rsync` idea
- Binaries

Simple meaning: Sprint 4 taught you how to turn Bash commands into a reusable script.

Sprint 5: Nano and Git Setup

You learned two important things:

- How to edit files in the terminal using Nano
- How to prepare Git correctly before starting real Git work

You learned about:

- Nano basics
- Save and exit in Nano
- What Git is
- What version control means
- What a commit is
- What a branch is
- Git vs GitHub
- Checking Git installation
- Setting `user.name`
- Setting `user.email`
- Setting `core.editor`
- Viewing Git config

Simple meaning: Sprint 5 prepared your terminal editing skill and Git identity setup.

2. What Sprint 6 Is About

Now you are starting the first real Git workflow sprint.

Sprint 6 is about:

- Creating a local Git repository
- Understanding what Git tracks
- Understanding the hidden `.git` folder
- Understanding file states
- Using the staging area
- Making commits
- Writing useful commit messages
- Viewing commit history using `git log`

Simple meaning: Sprint 6 teaches you how to track project changes on your own computer using Git.

3. Main Goal of Sprint 6

The main goal of Sprint 6 is:

Track local project changes with Git.

By the end of Sprint 6, you should be able to:

- Create a practice project
- Turn it into a Git repository

- Check the hidden `.git` folder
- Check Git status
- Create and edit files
- Understand untracked, modified, and staged files
- Stage one file
- Unstage one file
- Stage all intended files
- Make commits
- Understand commit messages
- Understand commit hashes
- View commit history using `git log`

Simple meaning: You should be able to use Git locally to save the history of your project changes.

4. Why Sprint 6 Is Important

Sprint 6 is important because Git is not useful only for installation and setup. Git becomes useful when you start tracking real project changes.

Later, you will use these Sprint 6 skills when working with:

- SQL files
- PostgreSQL project folders
- Bash scripts
- README files
- Real code projects
- GitHub repositories
- Team collaboration
- Pull requests
- Branches
- Code review

Before using GitHub, branches, remotes, and pull requests, you must first understand local Git.

Simple meaning: Sprint 6 gives you the basic Git workflow you will use again and again later.

Part 1: Core Git Concepts

5. What Is a Repository?

A **repository**, often called a **repo**, is a project folder that Git tracks.

Before Git:

```
my-project
```

This is only a normal folder.

After Git:

```
git init
```

That folder becomes a Git repository.

Simple meaning: A repository is a project folder with Git tracking turned on.

6. Local Repository

A **local repository** is a Git repository stored on your own computer.

In Sprint 6, you are only working locally. You are not sending anything to GitHub yet.

Simple meaning: A local repository is a Git-tracked project on your computer.

7. What Does `git init` Do?

The command:

```
git init
```

creates a new Git repository in the current folder.

It tells Git:

Start tracking this folder.

Example:

```
mkdir sprint6-practice  
cd sprint6-practice  
git init
```

Now `sprint6-practice` is a Git repository.

Simple meaning: `git init` starts Git tracking inside your current project folder.

8. What Is the Hidden `.git` Folder?

After you run:

```
git init
```

Git creates a hidden folder called:

```
.git
```

This folder stores Git's internal data. It contains the project history and tracking information.

Simple meaning: `.git` is the hidden folder where Git stores repository history and tracking data.

Very important:

Do not delete the `.git` folder.

If you delete `.git`, your normal files may still remain, but the Git history will be lost.

9. How to See Hidden Files

Because `.git` starts with a dot, it is hidden in many Unix-like systems. To show it, use:

```
ls -a
```

The `-a` flag means **all**. That includes hidden files.

Simple meaning: `ls -a` helps you see hidden files like `.git`.

10. What Is the Working Tree?

The **working tree** means the actual files and folders in your project that you can see and edit.

Example:

```
sprint6-practice/  
  README.md  
  index.html
```

These visible files are part of the working tree.

If you edit one of these files, the working tree changes.

Simple meaning: The working tree is your current project files.

11. Git Tracks Changes Over Time

Git is a version control system. That means it records project changes over time.

Example:

- Today you create `README.md`
- Tomorrow you add another line
- Later you fix a mistake

Git can help you:

- See what changed
- Save project history
- Compare versions
- Understand when changes happened

Simple meaning: Git remembers your project changes step by step.

Part 2: File States in Git

12. Why File States Matter

Git watches your files and gives them different states. These states help you understand what Git knows, what changed, and what will go into the next commit.

The most important states in Sprint 6 are:

- Untracked
- Modified
- Staged
- Committed

13. Untracked Files

An **untracked** file is a new file that Git has not started tracking yet.

Example: You create:

```
echo "Hello" > README.md
```

Then run:

```
git status
```

Git may show `README.md` as untracked.

That means:

- The file exists
- Git sees it
- But Git is not yet tracking it as part of history

Simple meaning: Untracked means the file is new and Git is not tracking it yet.

14. Modified Files

A **modified** file is a file Git already knows, but you changed it after the last commit.

Example:

- You commit `README.md`
- Then you add another line to it
- Git now sees it as modified

Simple meaning: Modified means Git already knows the file, but you changed it.

15. Staged Files

A **staged** file is a file selected for the next commit.

Git does not automatically put every changed file into the next commit. You choose which files should be included. That choosing step is called **staging**.

Simple meaning: Staged means the file is ready for the next commit.

16. Committed Files

A **committed** file is a file whose staged changes have already been saved into Git history.

After you run:

```
git commit -m "message"
```

Git saves those staged changes into a commit.

Simple meaning: Committed means the changes were saved in Git history.

17. Easy State Summary

State	Simple Meaning
Untracked	New file Git is not tracking yet
Modified	Git knows the file, but you changed it
Staged	File is selected for the next commit
Committed	File is saved in Git history

Part 3: Checking Repository Status

18. What Is `git status`?

The command:

```
git status
```

shows the current state of your repository.

It tells you:

- Which files are untracked
- Which files are modified

- Which files are staged
- Whether there is anything to commit

Simple meaning: `git status` tells you what is happening in your Git project right now.

19. Why `git status` Is Very Important

`git status` is one of the most useful beginner Git commands. You should use it again and again.

Use it:

- After creating files
- After editing files
- After staging files
- After unstaging files
- Before committing
- After committing

Simple meaning: `git status` helps you avoid confusion.

20. Good Beginner Habit

A very good beginner habit is:

```
git status
```

before and after every important Git action.

Simple meaning: Check first, act second.

Part 4: The Staging Area

21. What Is the Staging Area?

The **staging area** is the place where you prepare files for the next commit.

Imagine this situation:

- You changed 5 files
- But you only want to commit 2 of them

So you stage only those 2 files.

Simple meaning: The staging area is the waiting area for the next commit.

22. Why Git Uses a Staging Area

Git uses the staging area to give you control. You decide what should go into the next commit.

This is useful because:

- You can commit only one part of your work
- You can keep commits clean and organized
- You can avoid saving unfinished changes in the wrong commit

Simple meaning: The staging area helps you choose what Git should save next.

23. Staging One File with `git add`

To stage one file, use:

```
git add name-of-file
```

Example:

```
git add README.md
```

This tells Git:

Put `README.md` into the staging area for the next commit.

Simple meaning: `git add README.md` stages one file.

24. Staging All Changes with `git add .`

To stage many current changes at once, use:

```
git add .
```

The dot means the current folder and its changes.

Simple meaning: `git add .` stages many current changes in the current project.

Beginner warning: This can stage more than you expected. Always check with:

```
git status
```

before committing.

25. Unstaging a File with `git reset`

If you stage a file by mistake, you can remove it from the staging area.

Use:

```
git reset name-of-file
```

Example:

```
git reset README.md
```

This removes the file from the staging area.

Important:

- The file is not deleted
- Your file content is not lost
- Your changes still exist in the working tree

Simple meaning: `git reset README.md` unstages the file, but keeps your changes.

26. `git add` vs `git commit`

Many beginners confuse these two commands.

```
git add
```

This stages changes. It does **not** save history yet.

```
git commit
```

This saves staged changes into Git history.

Simple meaning:

- `git add` = prepare changes
 - `git commit` = save changes in history
-

Part 5: Commits

27. What Is a Commit?

A **commit** is a saved snapshot of your project.

It records the staged changes at a specific point in time.

Simple meaning: A commit is a saved checkpoint in Git history.

Example:

- Commit 1: added `README.md`
 - Commit 2: updated `README.md`
 - Commit 3: fixed spelling mistake
-

28. Why Commits Are Important

Commits are important because they:

- Save your progress
- Record the history of your work
- Help you understand what changed
- Help you go back and review older states

Simple meaning: Commits help you build your project history step by step.

29. What Is a Commit Message?

A **commit message** explains what changed in the commit.

Example:

```
git commit -m "docs: add README"
```

Here:

- `git commit` creates the commit
- `-m` means message
- The text explains the change

Simple meaning: A commit message is the explanation attached to a commit.

30. Why Good Commit Messages Matter

A good commit message helps you and other developers understand project history.

Bad message:

```
git commit -m "update"
```

This is too unclear.

Better message:

```
git commit -m "docs: add README"
```

This clearly explains the change.

Simple meaning: A good commit message should say what changed in a clear way.

31. Example Commit Messages

Good beginner examples:

```
git commit -m "feat: add homepage"  
git commit -m "fix: correct heading typo"  
git commit -m "docs: add project notes"  
git commit -m "docs: update README"
```

Simple meaning: Your commit message should be short, clear, and meaningful.

32. What Are Conventional Commits?

Conventional Commits are a clean commit message style. They use a type at the beginning of the message.

Examples:

- feat: add homepage
- fix: correct heading typo
- docs: add README
- refactor: improve file structure

Common types

- feat = new feature
- fix = bug fix
- docs = documentation change
- refactor = code improvement without changing behavior

Simple meaning: Conventional Commits are a clean way to write commit messages.

33. What Is a Commit Hash?

Every commit gets a unique ID called a **commit hash**.

When you run:

```
git log
```

Git shows each commit with its unique hash.

Example idea:

```
commit a3f5c91b7d9...
```

Simple meaning: A commit hash is the unique ID of a commit.

34. Why Commit Hashes Matter

Commit hashes matter because Git uses them to identify commits exactly.

They help Git:

- Tell commits apart
- Refer to a specific point in history
- Show exact commit records

Simple meaning: A commit hash helps Git know exactly which commit you mean.

Part 6: Viewing History with `git log`

35. What Is `git log`?

The command:

```
git log
```

shows the commit history of your repository.

It usually shows:

- Commit hash
- Author
- Date
- Commit message

Simple meaning: `git log` shows the saved history of your project.

36. What You Can Learn from `git log`

From `git log`, you can learn:

- What commits were made
- In what order they happened
- Who made them
- What message was used

Simple meaning: `git log` helps you read the story of your project.

37. How to Exit `git log`

When `git log` opens in a scrollable view, press:


```
q
```

Simple meaning: Press `q` to leave `git log`.

Part 7: Easy Git Workflow Idea

38. Think of Git Like a Camera

A very simple beginner idea is this:

Imagine your project is a room. You make changes inside the room. Before taking a photo, you choose what should appear in the photo. That choosing step is **staging**. Then Git takes the photo. That photo is the **commit**.

Simple flow:

```
Working tree → Staging area → Commit history
```

Another simple flow:

```
Edit files → Stage files → Commit files
```

Simple meaning: Git lets you choose changes first, then save them as a snapshot.

Part 8: Sprint 6 Practice Flow

39. Full Practice Task

Complete this practice slowly and carefully.

Step 1: Create a practice folder

```
mkdir sprint6-practice  
cd sprint6-practice
```

Meaning: You created a folder and moved into it.

Step 2: Start Git in that folder

```
git init
```

Meaning: This folder is now a Git repository.

Step 3: Show hidden files

```
ls -a
```

You should see:

```
.git
```

Meaning: Git created the hidden `.git` folder.

Step 4: Check repository status

```
git status
```

Meaning: Git shows the current repository condition.

Step 5: Create a file

```
echo "My Sprint 6 Git practice" > README.md
```

Meaning: You created a file called `README.md` with one line of text.

Step 6: Check status again

```
git status
```

You should now see `README.md` as **untracked**.

Meaning: Git sees the file, but is not yet tracking it in history.

Step 7: Stage the file

```
git add README.md
```

Meaning: You placed `README.md` into the staging area.

Step 8: Check status again

```
git status
```

Now `README.md` should appear as staged.

Meaning: The file is ready for the next commit.

Step 9: Unstage the file for practice

```
git reset README.md
```

Meaning: You removed the file from the staging area.

Step 10: Stage it again

```
git add README.md
```

Meaning: The file is again ready for the next commit.

Step 11: Make your first commit

```
git commit -m "docs: add README"
```

Meaning: You saved the first Git checkpoint.

Step 12: Edit the file again

```
echo "I am learning local Git workflow." >> README.md
```

Meaning: You added another line to the file.

Step 13: Check status again

```
git status
```

Now Git should show the file as **modified**.

Meaning: Git already knows the file, but you changed it after the previous commit.

Step 14: Stage the modified file

```
git add README.md
```

Meaning: The new changes are selected for the next commit.

Step 15: Make your second commit

```
git commit -m "docs: update README with Git workflow note"
```

Meaning: You saved the second Git checkpoint.

Step 16: View commit history

```
git log
```

You should see two commits.

To exit:

```
q
```

Meaning: You can now view the project history.

Part 9: Sprint 6 Command Summary

40. Sprint 6 Main Commands

Command	Simple Meaning
<code>git init</code>	Start Git in the current folder
<code>ls -a</code>	Show hidden files, including <code>.git</code>
<code>git status</code>	Show current Git state
<code>git add name-of-file</code>	Stage one file
<code>git add .</code>	Stage all current changes
<code>git reset name-of-file</code>	Unstage a file
<code>git commit -m "message"</code>	Save staged changes
<code>git log</code>	Show commit history

41. Most Important Local Git Workflow

Remember this cycle:

```
git status
git add file-name
git commit -m "message"
git log
```

Even simpler:

```
Check → Stage → Commit → View history
```

Simple meaning: This is the heart of Sprint 6.

Part 10: Common Beginner Mistakes

42. Mistake 1: Running `git init` in the Wrong Folder

Problem: You turn the wrong folder into a Git repository.

Better habit: First check where you are:

```
pwd
```

Then run:

```
git init
```

Simple meaning: Always make sure you are in the correct folder before starting Git.

43. Mistake 2: Forgetting `git status`

Problem: You do not understand what Git is tracking.

Better habit: Use:

```
git status
```

again and again.

Simple meaning: `git status` helps you understand what Git sees.

44. Mistake 3: Thinking `git add` Means Save

Problem: You think `git add` already saved your history.

Correct idea:

- `git add` stages changes
- `git commit` saves changes in Git history

Simple meaning: Stage first, commit second.

45. Mistake 4: Using Poor Commit Messages

Bad example:

```
git commit -m "stuff"
```

Better example:

```
git commit -m "docs: add README"
```

Simple meaning: A commit message should clearly say what changed.

46. Mistake 5: Deleting `.git`

Problem: You remove the hidden `.git` folder.

Result: The project files may still exist, but Git history is lost.

Simple meaning: Do not delete `.git`.

47. Mistake 6: Using `git add .` Without Checking

Problem: You stage more files than you wanted.

Better habit: Use:

```
git status
```

before and after `git add .`

Simple meaning: Check what you are staging.

48. Mistake 7: Confusing Untracked and Modified

Untracked

Git has never tracked the file before.

Modified

Git already tracked the file, but it changed after a commit.

Simple meaning: Untracked = new file. Modified = changed tracked file.

Part 11: Checkpoint Understanding

49. Sprint 6 Checkpoint

Your Sprint 6 checkpoint is:

Create a local repository, make two commits, and explain untracked vs modified vs staged.

A good beginner answer:

- **Untracked** means the file is new and Git is not tracking it yet.
- **Modified** means Git already knows the file, but I changed it.
- **Staged** means the file is selected and ready for the next commit.

Simple meaning: This checkpoint proves that you understand the basic local Git workflow.

Part 12: Key Terms Summary

50. Sprint 6 Key Terms

Term	Simple Meaning
Repository	A project folder tracked by Git
Local repository	Git project on your computer
<code>.git</code> folder	Hidden folder where Git stores history and tracking data
Working tree	The current visible project files
Untracked	New file Git is not tracking yet
Modified	Tracked file changed after the last commit
Staged	File selected for the next commit
Commit	Saved snapshot of staged changes
Commit message	Text explaining what changed in the commit

Term	Simple Meaning
Commit hash	Unique ID of a commit
Staging area	The place where files wait for the next commit
<code>git init</code>	Start a Git repository
<code>git status</code>	Show repository state
<code>git add</code>	Stage changes
<code>git reset name-of-file</code>	Unstage a file
<code>git log</code>	Show commit history

Part 13: Final Sprint 6 Understanding

51. What You Should Understand After Sprint 6

After Sprint 6, you should understand that Git is not only something you install and configure. It is a tool you use to track real project changes.

You should now understand:

- What a repository is
- What `git init` does
- What the hidden `.git` folder is
- Why `.git` is important
- What the working tree is
- What file states mean
- What untracked means
- What modified means
- What staged means
- What the staging area is
- What `git status` does
- What `git add` does
- What `git add .` does
- What `git reset name-of-file` does
- What a commit is
- What a commit message is
- What a commit hash is
- What `git log` does
- How local Git workflow works

Simple final meaning: Sprint 6 teaches you the first real Git workflow:

Create project → Start Git → Check status → Add files → Commit → View history

This is the foundation you need before moving to GitHub, remotes, branches, and pull requests.

Part 14: Sprint 6 Mini Assessment

52. Answer These Before Moving to Sprint 7

1. What is a Git repository?
2. What does `git init` do?
3. What is the `.git` folder?
4. Why should you not delete `.git`?
5. What is the working tree?
6. What does `git status` show?
7. What does untracked mean?
8. What does modified mean?
9. What does staged mean?
10. What does `git add README.md` do?
11. What does `git add .` do?
12. What does `git reset README.md` do?
13. What is a commit?
14. What is a commit message?
15. What is a commit hash?
16. What does `git log` show?
17. How do you exit `git log`?
18. What is a good commit message for adding a homepage?

Part 15: Sprint 6 Completion Checklist

53. Tick These When Completed

- ☐ I created a practice folder.
- ☐ I moved into the practice folder.
- ☐ I ran `git init`.
- ☐ I checked hidden files using `ls -a`.
- ☐ I saw the `.git` folder.
- ☐ I ran `git status`.
- ☐ I created a file such as `README.md`.
- ☐ I saw an untracked file in Git status.
- ☐ I staged a file using `git add README.md`.
- ☐ I unstaged a file using `git reset README.md`.

- ☐ I staged the file again.
 - ☐ I made my first commit.
 - ☐ I edited the file again.
 - ☐ I saw a modified file in Git status.
 - ☐ I staged the modified file.
 - ☐ I made my second commit.
 - ☐ I ran `git log`.
 - ☐ I can explain untracked, modified, and staged in simple words.
 - ☐ I understand the local Git workflow.
-

End of Sprint 6 Notes

You are ready to move to **Sprint 7** only after you can:

- Create a local repository
- Make two commits
- Use `git status` confidently
- Explain untracked, modified, staged, and commit in simple words